

Sleep Study Guard 프로젝트 결과 보고서

공부 중 졸음 감지 및 깨움 웹 서비스

제출 전 입력 필요: GitHub Repository 링크 / 실행 시연 영상 링크 / 팀원명

작성일: 2026년 6월

1. 프로젝트 개요

본 프로젝트는 공부 중 사용자가 잠에 드는 상황을 웹캠 기반으로 감지하고, 졸음으로 판단될 경우 화면 상태 표시와 알림음으로 사용자를 깨워주는 웹 서비스이다. 프론트엔드는 React, 백엔드는 FastAPI, 데이터베이스는 SQLAlchemy ORM 기반 SQLite를 사용하였고, 머신러닝 모델은 scikit-learn의 RandomForestClassifier를 사용하였다.

서비스의 핵심 흐름은 사용자가 공부 세션을 시작하면 웹캠 프레임을 주기적으로 캡처하고, 백엔드에서 MediaPipe FaceMesh로 얼굴 랜드마크와 눈 주변 좌표를 추출한 뒤 EAR(Eye Aspect Ratio), 눈 감김 지속 시간, 얼굴 기울기 특징값을 계산하여 awake/drowsy 상태를 예측하는 방식이다.

2. 주제 선정 이유

공부 중 졸음은 학습 효율을 급격히 떨어뜨리지만, 사용자는 졸음이 누적되는 순간을 스스로 인지하기 어렵다. 특히 시험 기간이나 온라인 강의 수강 상황에서는 잠깐의 졸음이 긴 시간의 학습 공백으로 이어질 수 있다.

발견한 페인포인트는 다음과 같다. 첫째, 학생은 공부 시간을 기록하더라도 실제 집중 상태를 함께 기록하지 못한다. 둘째, 스마트폰 알람이나 타이머는 시간이 되었을 때만 울릴 뿐 사용자의 생체 상태를 반영하지 않는다. 셋째, 졸음 빈도와 시간대를 데이터로 남기지 않으면 자신의 학습 패턴을 개선하기 어렵다.

따라서 본 프로젝트는 단순 공부 타이머를 넘어 웹캠 기반 졸음 감지, 즉시 알림, 세션별 졸음 로그 저장, 통계 조회를 결합하여 학습 집중도를 보조하는 서비스를 목표로 하였다.

3. 이론적 배경 및 유사 서비스 분석

졸음 감지는 일반적으로 눈 깜빡임, 눈 감김 지속 시간, 얼굴 방향 변화, 고개 숙임 등을 조합하여 판단한다. 본 프로젝트에서는 얼굴 랜드마크 기반 EAR을 주요 특징으로 사용하였다. EAR은 눈의 수직 거리와 수평 거리의 비율을 계산한 값으로, 눈을 감으면 수직 거리가 줄어들어 값이 낮아지는 특성이 있다.

유사 사례로는 운전자 졸음 감지 시스템, 스마트워치 수면 감지, 공부 타이머 앱이 있다. 운전자 졸음 감지 시스템은 얼굴과 눈 상태를 분석한다는 점에서 유사하지만 차량 환경에 집중되어 있고 학습 세션 기록 기능은 부족하다. 일반 공부 타이머 앱은 공부 시간 기록에는 강하지만 사용자의 졸음 상태를 직접 감지하지 못한다.

본 서비스의 차별점은 웹 브라우저에서 바로 사용할 수 있는 공부 세션 중심 서비스라는 점이다. 사용자는 별도 하드웨어 없이 노트북 웹캠만으로 졸음 상태를 확인할 수 있고, 졸음 감지 로그가 공부 세션과 연결되어 저장되므로 이후 집중 패턴 분석에 활용할 수 있다.

4. 시스템 아키텍처

전체 구조는 React 프론트엔드, FastAPI 백엔드, SQLite 데이터베이스, modeling 폴더의 모델 학습 파이프라인으로 구성된다. React는 웹캠 스트림을 표시하고 주기적으로 프레임을 캡처하여 백엔드에 전송한다. FastAPI는 이미지 프레임을 디코딩하고 MediaPipe/OpenCV로 특징값을 추출한 뒤 ML 모델에 입력한다.

백엔드는 예측 결과를 공부 세션과 연결된 졸음 로그로 저장한다. 모델링 코드는 CSV 형태의 특징 데이터에서 RandomForest 모델을 학습하고 joblib 파일로 export한다. 런타임에서는 export된 모델을 로드하여 실시간 예측에 사용한다.

5. 사용 기술 스택

Frontend: React, Vite, WebRTC getUserMedia API, Canvas overlay

Backend: FastAPI, Pydantic, SQLAlchemy ORM, SQLite, CORS Middleware

Vision/ML: OpenCV, MediaPipe FaceMesh, scikit-learn RandomForestClassifier, joblib

Modeling: pandas 기반 CSV 로딩, train/test split, StandardScaler + RandomForest 파이프라인

Project structure: frontend, backend, modeling 세 디렉토리로 역할 분리

6. 데이터 수집 및 활용

웹캠 얼굴 인식 데이터는 React 화면에서 사용자의 웹캠 프레임을 주기적으로 캡처하여 FastAPI로 전송하는 방식으로 수집한다. 서버는 프레임에서 얼굴 위치, 눈 주변 랜드마크, 얼굴 방향 정보를 추출한다.

졸음 상태 분류 데이터는 EAR, 눈 감김 지속 시간, 얼굴 pitch/yaw/roll 등의 특징값과 awake(1), drowsy(0) 라벨로 구성된다. 본 프로젝트의 modeling/data/drowsiness_samples.csv는 시연 가능한 소규모 예시 데이터이며, 실제 서비스 정확도 향상을 위해서는 사용자별 라벨링 데이터를 추가로 확보해야 한다.

공부 세션 데이터는 사용자가 시작/종료 버튼을 누를 때 subject, started_at, ended_at, duration_seconds를 저장한다. 졸음 감지 로그는 예측 시각, 졸음 여부, 예측 확률, 특징값을 study_session과 연결하여 저장한다.

7. 계획 단계 산출물

본 프로젝트는 계획 단계 산출물로 ERD(DBML), 시퀀스 다이어그램, 유스케이스 다이어그램을 작성하였다. ERD는 실제 backend/dbml/schema.dbml 파일로 관리하며, 아래 내용은 보고서용 요약이다.

7.1 ERD(DBML)

```
Table users {
  id integer [pk, increment]
  name varchar(80) [not null]
  created_at datetime [not null]
}
Table study_sessions {
  id integer [pk, increment]
  user_id integer [not null, ref: > users.id]
  subject varchar(120) [not null]
  started_at datetime [not null]
  ended_at datetime
  duration_seconds integer
  is_active boolean [not null]
}
Table drowsiness_logs {
  id integer [pk, increment]
  session_id integer [not null, ref: > study_sessions.id]
  detected_at datetime [not null]
  is_drowsy boolean [not null]
  probability float [not null]
  ear float
  eye_closed_duration_ms integer
  face_pitch float
  face_yaw float
  face_roll float
}
```

7.2 시퀀스 다이어그램(Mermaid)

```
sequenceDiagram
    actor User as User
    participant FE as React Frontend
    participant API as FastAPI Backend
    participant ML as ML Predictor
    participant DB as SQLite DB
    User->>FE: 
    FE->>API: POST /study-sessions/start
    API->>DB: StudySession 
    FE->>API: POST /drowsiness/predict-frame
    API->>API: FaceMesh 
    API->>ML: EAR/ / 
    ML->>API: awake drowsy + probability
    API->>DB: DrowsinessLog 
    API->>FE: bounding box 
    FE->>User: ,
```

7.3 유스케이스 다이어그램(Mermaid)

```
graph LR
    User((User)) --> Start[Start]
    Start --> Camera[Camera]
    Camera --> Predict[Predict]
    Predict --> Alert[Alert]
    Alert --> End[End]
    End --> Stats[Stats]
    Stats --> User
```

8. 구현 결과

구현 결과물은 사용자가 브라우저에서 공부 과목을 입력하고 세션을 시작하면 웹캠 영상과 인식 bounding box가 표시되는 형태이다. 백엔드는 /drowsiness/predict-frame API를 통해 이미지 프레임을 수신하고, 얼굴/눈 랜드마크에서 특징값을 계산한 뒤 ML 모델로 졸음 여부를 분류한다.

GitHub Repository 링크: 제출 전 입력 필요

실행 시연 영상 링크: 제출 전 입력 필요

8.1 핵심 API

Endpoint	Method	역할
/health	GET	서버 상태 확인
/debug/runtime	GET	Python/MediaPipe 런타임 확인
/users/default	GET	기본 사용자 생성 또는 조회
/study-sessions/start	POST	공부 세션 시작 및 DB 저장
/study-sessions/{id}/end	POST	공부 세션 종료 및 공부 시간 계산
/drowsiness/predict	POST	특징값 기반 졸음 예측
/drowsiness/predict-frame	POST	웹캠 프레임 기반 특징 추출 및 졸음 예측
/drowsiness/logs	GET	세션별 졸음 감지 로그 조회
/stats/session/{id}	GET	세션 통계 조회

9. 트러블 슈팅

문제 1: CORS 오류처럼 보였지만 실제 원인은 서버 500 오류

React에서 /drowsiness/predict-frame 호출 시 브라우저 콘솔에는 CORS 오류가 표시되었다. 서버 로그를 확인한 결과 실제 원인은 MediaPipe import 이후 mp.solutions.face_mesh가 존재하지 않아 AttributeError가 발생한 500 오류였다. 브라우저는 500 응답에 CORS 헤더가 붙지 않자 CORS 문제처럼 표시했다.

해결 과정은 서버 로그 추적, 예외 처리 보강, CORS origin regex 추가, /debug/runtime endpoint 추가 순서로 진행했다. 특히 현재 Python 3.14 환경에서 설치된 최신 mediapipe는 mp.solutions를 제공하지 않았기 때문에 Python 3.11 가상환경(.venv311)을 만들고 mediapipe==0.10.14로 버전을 고정하였다. 이후 /debug/runtime에서 python=3.11.15, mediapipe=0.10.14, has_solutions=true를 확인하였다.

문제 2: 눈을 감아도 EAR 값이 거의 변하지 않음

초기 구현에서는 MediaPipe가 정상 동작하지 않아 OpenCV Haar cascade fallback으로 눈 영역을 추정했다. 이 방식은 감은 눈의 속눈썹 선이나 그림자를 열린 눈처럼 해석하여 EAR이 0.29~0.34 근처에서 유지되는 문제가 있었다. 또한 React setInterval 내부에서 closedSince state의 오래된 값을 참조하여 눈 감김 지속 시간이 누적되지

않는 문제가 있었다.

해결 과정은 두 단계로 이루어졌다. 먼저 React의 closedSince를 useRef로 변경하여 interval 내부에서도 최신 감김 시작 시각을 참조하게 했다. 다음으로 Python 3.11 + mediapipe==0.10.14 환경을 구성해 FaceMesh 기반 실제 얼굴 랜드마크로 EAR을 계산하도록 복구했다. 그 결과 OpenCV fallback보다 안정적인 눈 좌표 기반 판단이 가능해졌다.

10. 생성형 AI 활용 내역

본 프로젝트에서는 ChatGPT를 설계, 구현, 오류 분석, 보고서 초안 작성에 활용하였다. 제출 시 실제 대화 화면 캡처를 첨부하거나, 아래 표의 핵심 프롬프트를 캡처 자료와 대응시키면 된다.

활용 목적	핵심 프롬프트 요약
프로젝트 기획 및 구조 설계	FastAPI + SQLAlchemy ORM + ML 조건을 만족하는 졸음 감지 웹 프로젝트를 frontend/backend/modeling 구조로 설계해줘.
DB/ERD 설계	공부 세션 데이터와 졸음 감지 로그 데이터를 저장하기 위한 1:N 관계 테이블과 DBML을 작성해줘.
트러블슈팅	React에서 predict-frame 요청 시 CORS 오류와 500 오류가 발생한다. 서버 로그를 보고 원인을 찾아 수정해줘.
MediaPipe 호환성 해결	MediaPipe FaceMesh가 제대로 되는 Python 3.10 또는 3.11 가상환경으로 맞춰줘.
보고서 작성	제출 안내 항목에 맞춰 프로젝트 결과 보고서를 작성하고 PPT 제작 AI에게 넘길 프롬프트를 만들어줘.

11. 배운 점 및 느낀 점

이번 프로젝트를 통해 단순히 모델을 학습시키는 것보다 모델에 입력되는 특징값의 품질이 더 중요하다는 점을 배웠다. 눈을 감았는데 EAR이 변하지 않으면 모델은 졸음을 맞출 수 없으므로, 컴퓨터 비전 전처리와 랜드마크 추출이 ML 성능의 핵심이라는 것을 체감했다.

또한 브라우저 콘솔에 표시되는 오류가 실제 원인을 그대로 보여주지 않을 수 있다는 점을 배웠다. CORS 오류처럼 보였지만 실제로는 서버 내부 500 오류였고, 서버 로그를 함께 확인해야 정확한 원인을 찾을 수 있었다.

개인 회고: 프로젝트 초반에는 기능 구현에만 집중했지만, 후반에는 실행 환경, 라이브러리 버전, 사용자 피드백, 디버깅 화면의 중요성을 느꼈다. 특히 bounding box 표시를 추가하면서 AI/ML 기능도 사용자가 이해할 수 있게 시각화해야 신뢰할 수 있다는 점을 배웠다.

12. 참고문헌 출처

FastAPI 공식 문서. <https://fastapi.tiangolo.com/>

SQLAlchemy ORM 공식 문서. <https://docs.sqlalchemy.org/en/20/orm/>

scikit-learn RandomForestClassifier 공식 문서.

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

OpenCV Cascade Classifier Tutorial. https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html

Lugaresi et al. MediaPipe: A Framework for Building Perception Pipelines. <https://arxiv.org/abs/1906.08172>

Grishchenko et al. Attention Mesh: High-fidelity Face Mesh Prediction in Real-time.

<https://arxiv.org/abs/2006.10962>

kairess eye_blink_detector dataset. https://github.com/kairess/eye_blink_detector/tree/master/dataset